

Ingeniería de Software 3- Actualidad Informática

Trabajo Práctico Nro. 4 (2026) - Análisis y Gestión de Riesgos en el Desarrollo de Software

Alumnos:

- Navarro, Eduardo
- Proscopio Diaz, Karen
- Samudio, Paola
- Cotrina Wilches, Juan Diego

URL del repositorio remoto: [Ed2128/TPS-IS3: Repositorio de los trabajos prácticos de Ingeniería de Software 3, 2026.](#)

[SEI] Análisis y Gestión de Riesgos

El presente análisis de riesgo ha sido confeccionado para el proyecto de desarrollo del **Sistema de Gestión de Eventos Académicos (SGEA)**. En particular, se ha optado por la evaluación de [objetivos]. El mismo ha sido confeccionado siguiendo los lineamientos de la metodología Software Risk Management (SRM) del Software Engineering Institute (SEI), es en base a esta selección que se ha estructurado el contenido del presente documento.

El proyecto surge ante la necesidad de contar con una aplicación web accesible desde cualquier dispositivo que facilite a grupos de personas la organización y administración de eventos de tipo académico, tales como cursos, jornadas, congresos y charlas. El desarrollo es ejecutado por el equipo técnico asignado, enfocándose en resolver módulos críticos de negocio como la gestión de eventos, inscripciones autónomas, manejo de roles, acreditación de participantes y generación automática de informes y certificados en formato PDF. El proceso abarca desde el relevamiento inicial de requerimientos mediante prácticas Spec-Driven Development (SDD) hasta la implantación final de la solución dentro de un plazo cronológico acotado.

Inventario de activos a proteger

- Entornos locales de desarrollo.
- Código fuente del producto en desarrollo en GitHub.
- Documentación técnica y especificaciones.
- Instancia de la base de datos PostgreSQL de desarrollo.
- Miembros del equipo de desarrollo.
- Plazo temporal asignado al proyecto.
- Entre otros

Objetivos

- Evaluar los riesgos relacionados al desarrollo del software SGEA.
- Definir estrategias vinculadas a la gestión de los riesgos detectados, priorizando aquellos que sean identificados como críticos a raíz del presente análisis.
- Definir planes de contingencia para aquellas situaciones en las que uno o más riesgos puedan convertirse en problemas y cuyo impacto deba ser minimizado.

Equipo de trabajo

- Navarro, Eduardo
- Proscopio Diaz, Karen
- Samudio, Paola
- Cotrina Wilches, Juan Diego

Taxonomía de los riesgos

Identificador	Elemento	Riesgo	Vinculación del riesgo
R1	Equipo de desarrollo	Falta de experiencia práctica en la integración de Next.js con el backend Spring Boot.	Proyecto, Producto
R2	Base de Datos	Caída o pérdida de datos en la instancia de PostgreSQL de desarrollo.	Proyecto
R3	Módulo de Seguridad	Vulnerabilidades de “Broken Access Control” en la asignación de roles.	Producto, Negocio
R4	Stakeholders	Falta de disponibilidad de los organizadores para validar las reglas de negocio de los eventos.	Proyecto, Negocio
R5	Código Fuente	Conflictos severos al fusionar ramas (Merge Conflicts) por mala coordinación en GitHub	Proyecto

Declaración de los riesgos

R1 - Falta de experiencia práctica en la integración de Next.js con el backend Spring Boot	
Condición	El equipo no cuenta con experiencia práctica en la integración de Next.js con Spring Boot, lo cual implica configuraciones de CORS y manejos de tokens que requieren una curva de aprendizaje.
Consecuencia	El equipo deberá invertir tiempo del desarrollo en investigar, capacitarse y realizar desarrollos de prueba y error para lograr la comunicación correcta entre ambas partes.
Efecto	Retrasos en la estimación de plazos iniciales del proyecto. Además, podrían generarse deficiencias en el rendimiento de las peticiones HTTP por no aplicar las mejores prácticas desde el comienzo.

R2 - Caída o pérdida de datos en la instancia de PostgreSQL en el desarrollo	
Condición	La base de datos de desarrollo se ejecuta de forma local en los entornos individuales de los integrantes del grupo sin una estrategia unificada de respaldos automatizados o un control formal de versiones de la estructura de datos (como scripts de migración).
Consecuencia	Ante un fallo del sistema operativo, corrupción del servicio o un error humano accidental, los datos de prueba y la estructura de las tablas se pierden, obligando al equipo a reconstruir el esquema manualmente desde cero.
Efecto	Pérdida de tiempo valioso en la reconfiguración técnica del entorno, retrasos en la ejecución de las pruebas locales de integración y riesgo de inconsistencias en los modelos de datos entre los distintos desarrolladores.

R3 - Vulnerabilidades de “Broken Access Control” en la asignación de roles	
Condición	Los endpoints de asignación de roles o creación de eventos podrían programarse asumiendo que ocultar elementos en la interfaz (Next.js) es suficiente, omitiendo la validación estricta del token JWT a nivel de controlador en Spring Security.
Consecuencia	Usuarios con rol básico de “Participante” podrían interceptar y forzar peticiones HTTP directas a la API logrando ejecutar acciones no permitidas, como auto-asignarse el rol de “Organizador”.
Efecto	Compromiso total de la confidencialidad e integridad del producto. El sistema permitiría la creación de eventos falsos y el acceso a datos sensibles (como DNI o correos), lo que invalidaría la viabilidad del software para el negocio.

R4 - Falta de disponibilidad de los stakeholders para validar las reglas de negocio de los eventos	
Condición	Los organizadores de eventos ,o la cátedra que actúa como cliente final, disponen de tiempos limitados para reuniones, lo que limita los espacios para la validación de los requerimientos.
Consecuencia	El equipo de desarrollo se ve obligado a tomar decisiones de diseño e implementar lógicas de negocio complejas basándose en suposiciones propias sin confirmación local.
Efecto	El producto desarrollado podría no cumplir con las expectativas o necesidades reales del negocio, forzando a realizar reestructuraciones complejas y refactorizaciones de código en etapas avanzadas del ciclo de vida, lo que incrementaría drásticamente el esfuerzo y los plazos de entrega.

R5 - Conflictos severos al fusionar ramas (Merge Conflicts) por mala coordinación en GitHub	
Condición	El equipo trabaja en paralelo sobre módulos de código que están directamente conectados, sin una política estricta de sincronización diaria ni una delimitación clara de tareas por archivo.
Consecuencia	Al momento de unificar las ramas de trabajo individuales en la rama main, Git detecta modificaciones superpuestas en las mismas líneas de código, lo que impide la fusión automática y genera conflictos complejos.
Efecto	Bloqueos temporales en el flujo de trabajo del equipo, retrasos en la consolidación de una versión estable y funcional del sistema, y el riesgo de sobrescribir accidentalmente código válido de un compañero durante la resolución manual de un conflicto.

Estimación de la probabilidad

Rango de probabilidad	Promedio para el cálculo	Expresión de lenguaje natural	Valor numérico
de 1% a 10%	5%	Baja	1
de 11 % a 25%	18%	Poco probable	2
de 26% a 55%	40%	Media	3
de 56% a 80%	68%	Altamente probable	4
de 81% a 99%	90%	Casi seguro	5

Estimación de probabilidad para cada riesgo:

Identificador	Elemento	Expresión	Probabilidad
R1	Falta de experiencia práctica en la integración de Next.js con el backend Spring Boot	Altamente probable	70%
R2	Caída o pérdida de datos en la instancia de PostgreSQL en el desarrollo	Poco probable	20%
R3	Vulnerabilidades de “Broken Access Control” en la asignación de roles	Media	50%
R4	Falta de disponibilidad de los stakeholders para validar las reglas de negocio de los eventos	Poco probable	20%
R5	Conflictos severos al fusionar ramas (Merge Conflicts) por mala coordinación en GitHub	Media	40%

Estimación del impacto

Criterio	Período en el que el proyecto se verá afectado	Valor numérico
Insignificante	1 día	1
Marginal	3 días	2
Medio	1 semana	3
Crítico	2 semanas	4
Catastrófico	4 semanas	5

Estimación del impacto para cada riesgo

Identificador	Riesgo	Impacto
R1	Falta de experiencia práctica en la integración de Next.js con el backend Spring Boot	Crítico
R2	Caída o pérdida de datos en la instancia de PostgreSQL en el desarrollo	Medio
R3	Vulnerabilidades de “Broken Access Control” en la asignación de roles	Catastrófico
R4	Falta de disponibilidad de los stakeholders para validar las reglas de negocio de los eventos	Marginal
R5	Conflictos severos al fusionar ramas (Merge Conflicts) por mala coordinación en GitHub	Medio

Magnitud de exposición al riesgo

Aprox. 1 = bajo riesgo.

Aprox. 2 = riesgo medio.

Aprox. 3 = alto riesgo

Identificador	Riesgo	Impacto	Probabilidad	Exposición
R1	Falta de experiencia práctica en la integración de Next.js con el backend Spring Boot	Crítico	70%	$4 * 0.7 = 2.8$
R2	Caída o pérdida de datos en la instancia de PostgreSQL en el desarrollo	Medio	20%	$3 * 0.2 = 0.6$
R3	Vulnerabilidades de “Broken Access Control” en la asignación de roles	Catastrófico	50%	$5 * 0.5 = 2.5$
R4	Falta de disponibilidad de los stakeholders para validar las reglas de negocio de los eventos	Marginal	20%	$2 * 0.2 = 0.4$
R5	Conflictos severos al fusionar ramas (Merge Conflicts) por mala coordinación en GitHub	Medio	40%	$3 * 0.4 = 1.2$

Planes de gestión de los riesgos

La gestión de los riesgos se reconoce como un proceso continuo, por lo que el presente documento podría ser adaptado a medida que se avanza con su ejecución.

Se presentan los planes de acción (preventivos) y de contingencias (reactivos) para los riesgos cuya exposición fuera superior a los umbrales definidos en el análisis precedente.

1. R1 - Falta de experiencia práctica en la integración de Next.js con el backend Spring Boot

1.1. Descripción de aspectos principales

Importancia del riesgo	Si este riesgo se dispara, implicaría que el equipo de desarrollo se enfrentará a bloqueos técnicos severos al intentar conectar la interfaz de usuario con la lógica del servidor. Esto impedirá visualizar las funcionalidades del sistema en el frontend, afectando directamente los plazos de entrega comprometidos para el proyecto.
Información requerida para su seguimiento	• Estado de las pruebas de conectividad entre el frontend y los endpoints del backend. • Registro de errores de CORS en la consola del navegador durante el desarrollo. • Tiempo de demora reportado en las tareas de consumo de APIs.
Responsable	Responsable del proyecto
Recursos necesarios	• Documentación técnica oficial de Next.js y Spring Security • Tiempo asignado del equipo para pruebas iniciales de conectividad • Entornos de desarrollo correctamente configurados

1.2. Plan de acción

- 1.2.1. Determinar el estado actual de las capacidades del equipo respecto a la integración de Next.js y Spring Boot mediante una evaluación interna.
- 1.2.2. Determinar y priorizar las necesidades de investigación o capacitación específicas requeridas para el desarrollo del proyecto.
- 1.2.3. Buscar alternativas de capacitación en caso de que el equipo requiera reforzar estos conocimientos.
- 1.2.4. Diseñar un plan de pruebas de integración temprana, destinando los recursos y el tiempo necesarios en el cronograma para alcanzar los objetivos del proyecto.
- 1.2.5. Establecer cuáles integraciones son estrictamente necesarias para una versión inicial del producto y cuáles podrían sumarse posteriormente.

1.3. Plan de contingencias

1.3.1. **Disparador:** El equipo declara experimentar un bloqueo técnico o una desviación en los tiempos estimados para la integración entre Next.js y Spring Boot que supere el límite establecido en la planificación:

1.3.1.1. En función de lo declarado por el equipo, evaluar el alcance de la problemática técnica detectada para priorizar aquellos componentes de la integración que deban ser resueltos inicialmente.

1.3.1.2. Seleccionar una o más de las alternativas de investigación o asistencia técnica identificadas previamente durante el proceso de planificación.

1.3.1.3. Implementar el plan de pruebas de integración temprana y reasignar los esfuerzos en el cronograma para mitigar el impacto en el avance del proyecto.

2. R3 - Vulnerabilidades de "Broken Access Control" en la asignación de roles

2.1. Descripción de aspectos principales

Importancia del riesgo	Si este riesgo se dispara, implicaría que la codificación o la configuración de seguridad del producto no validaría adecuadamente las autorizaciones de los usuarios. Esto permitiría accesos no autorizados a funciones críticas (como la asignación de roles o alteración de eventos), comprometiendo la confidencialidad e integridad del sistema y afectando la viabilidad del software para el negocio.
Información requerida para su seguimiento	• Requerimientos relacionados a los aspectos de seguridad y restricciones de acceso a nivel de endpoints. • Información de las evaluaciones de vulnerabilidades y revisiones de código realizadas periódicamente.
Responsable	Responsable del área de desarrollo.
Recursos necesarios	• Equipo de desarrollo. • Software o herramientas para la evaluación de cumplimiento de reglas de seguridad en el código fuente (estándares OWASP).

2.2 Plan de acción

2.2.1. Establecer si se deberá seguir un conjunto de prácticas de codificación de seguridad específicas (como los controles OWASP) por requerimientos internos o externos del proyecto.

2.2.2. Definir un conjunto de reglas de control de acceso que se pasen a considerar el estándar de seguridad a seguir por los desarrolladores al escribir el código fuente y configurar las rutas del sistema.

2.2.3. Seleccionar y configurar una metodología de revisión o herramienta de software que automatice la verificación de las cuestiones establecidas en el estándar de control de accesos.

2.2.4. Diseñar estrategias para asegurar que las restricciones de seguridad se mantengan vigentes ante futuras integraciones de nuevos componentes o módulos en el producto.

2.2.5. Determinar las acciones a seguir ante la detección de desvíos con respecto a las reglas de autorización establecidas en el documento de estándares.

2.3. Plan de contingencias

2.3.1. **Disparador:** Se detecta que el código fuente o los endpoints configurados no siguen los lineamientos de control de acceso definidos o presentan fallas en las restricciones de autorización:

2.3.1.1. Implementar alguna de las acciones previstas para solucionar los desvíos detectados en las políticas de control de acceso del código fuente.

2.3.1.2. Ejecutar nuevamente los controles de validación de seguridad y verificar que las restricciones han sido aplicadas en el sistema sin inconvenientes.

Anexo: Enriquecimiento de Specs (OWASP):

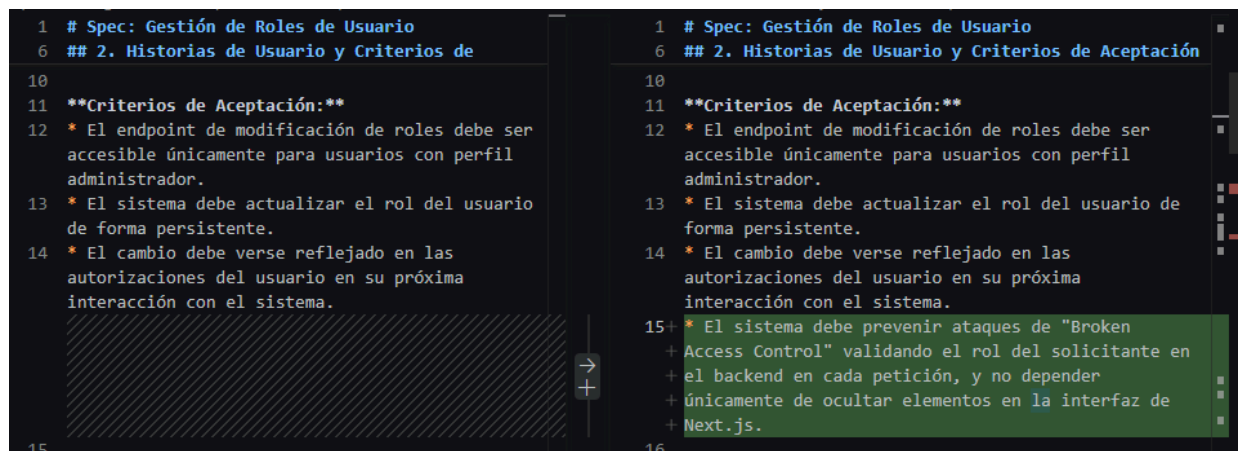
A continuación, se adjunta el paso a paso de la modificación realizada en el repositorio de GitHub sobre el archivo “gestion_rols_spec.md”. Se incorporaron controles de seguridad derivados de OWASP (prevención de Broken Access Control y Mass Assignment) para mitigar el riesgo técnico R3, de acuerdo a lo planteado en el plan de gestión.

1. Sincronizamos con el repositorio remoto y luego creamos una rama nueva.

```
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git pull origin main
remote: Enumerating objects: 37, done.
remote: Counting objects: 100% (37/37), done.
remote: Compressing objects: 100% (19/19), done.
remote: Total 33 (delta 16), reused 25 (delta 11), pack-reused 0 (from 0)
Unpacking objects: 100% (33/33), 9.04 KiB | 23.00 KiB/s, done.
From https://github.com/Ed2128/TPS-IS3
* branch          main          -> FETCH_HEAD
  9cc01db..52e76b2 main          -> origin/main
Updating 9cc01db..52e76b2
Fast-forward
 .vs/slnx.sqlite      | Bin 0 -> 90112 bytes
 README.md            |   3 +++
 specs/Incripcion_Participantes.md |   4 +++-
 3 files changed, 6 insertions(+), 1 deletion(-)
 create mode 100644 .vs/slnx.sqlite
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git checkout -b tp4-seguridad-roles
Switched to a new branch 'tp4-seguridad-roles'
```

2. Modificamos el archivo “specs/gestion_rols_spec.md” agregándole los controles de seguridad OWASP.

En criterios de aceptación:



Line	Original Text	Modified Text
1	# Spec: Gestión de Roles de Usuario	# Spec: Gestión de Roles de Usuario
6	## 2. Historias de Usuario y Criterios de	## 2. Historias de Usuario y Criterios de Aceptación
10		
11	**Criterios de Aceptación:**	**Criterios de Aceptación:**
12	* El endpoint de modificación de roles debe ser accesible únicamente para usuarios con perfil administrador.	* El endpoint de modificación de roles debe ser accesible únicamente para usuarios con perfil administrador.
13	* El sistema debe actualizar el rol del usuario de forma persistente.	* El sistema debe actualizar el rol del usuario de forma persistente.
14	* El cambio debe verse reflejado en las autorizaciones del usuario en su próxima interacción con el sistema.	* El cambio debe verse reflejado en las autorizaciones del usuario en su próxima interacción con el sistema.
15		+ El sistema debe prevenir ataques de "Broken Access Control" validando el rol del solicitante en el backend en cada petición, y no depender únicamente de ocultar elementos en la interfaz de Next.js.
16		

En restricciones técnicas y específicas de este módulo:

<pre>1 # Spec: Gestión de Roles de Usuario 21 ## 4. Restricciones técnicas específicas de 22 23- ***Seguridad:** Implementar Spring Security - para interceptar la ruta de actualización y - requerir explícitamente el rol `ADMIN`. 24 25 ***Arquitectura:** Respetar la separación por capas mediante `UsuarioController`, `UsuarioService` y `UsuarioRepository`. 26 27 ***Manejo de Errores:** En caso de que un usuario no autorizado intente acceder, el `GlobalExceptionHandler` debe gestionar la respuesta y retornar un HTTP 403 Forbidden. 28 29 ***Base de Datos:** El rol se almacenará en la columna `rol` de la tabla `usuarios`, mapeado preferentemente como un Enum de Java. 30 31 ## 5. Modelo de datos de este módulo</pre>	<pre>1 # Spec: Gestión de Roles de Usuario 22 ## 4. Restricciones técnicas específicas de este 23 24+ ***Seguridad:** Implementar Spring Security para + interceptar la ruta de actualización y requerir + explícitamente el rol `ADMIN` mediante anotaciones + @PreAuthorize. 25 26 ***Arquitectura:** Respetar la separación por capas mediante `UsuarioController`, `UsuarioService` y `UsuarioRepository`. 27 28 ***Manejo de Errores:** En caso de que un usuario no autorizado intente acceder, el `GlobalExceptionHandler` debe gestionar la respuesta y retornar un HTTP 403 Forbidden. 29 30 ***Base de Datos:** El rol se almacenará en la columna `rol` de la tabla `usuarios`, mapeado preferentemente como un Enum de Java. 31 32 ***Prevención Inyección de Datos:** Prevenir la + vulnerabilidad de Asignación Masiva (Mass + Assignment) asegurando que el UsuarioUpdateDTO solo + exponga el campo `rol` y no permita alterar + contraseñas o IDs en la misma petición. 33 34 ## 5. Modelo de datos de este módulo</pre>
---	---

3. Commiteamos el cambio:

```
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git add .
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git commit -m "Se enriqueció la spec de gestión de roles con controles OWASP para mitigar el riesgo 3"
[tp4-seguridad-roles 9e56c5a] Se enriqueció la spec de gestión de roles con controles OWASP para mitigar el riesgo 3
1 file changed, 3 insertions(+), 2 deletions(-)
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3>
```

4. Subimos la rama al repositorio remoto en GitHub:

```
PS C:\Users\edujn\Desktop\2026\Ing3\TPS-IS3> git push -u origin tp4-seguridad-roles
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 738 bytes | 184.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack reused 0 (from 0)
```

5. Creamos una pull request:



Add a title *

Se enriqueció la spec de gestión de roles con controles OWASP

Add a description

WritePreview

H B I E < > Link List Bulleted List Numbered List Quote Code Block

Se enriqueció la spec de gestión de roles con controles OWASP para mitigar el Riesgo 3: Vulnerabilidades de "Broken Access Control" en la asignación de roles.

Markdown is supported Paste, drop, or click to add files

Create pull request